

Hermes Meta-Intelligence Architecture

Master Agent + Industry Specialist Agents + Free-Tier Model Routing

Vision: A self-improving, cost-free, hallucination-safe agentic system

Generated for client development work | Confidential

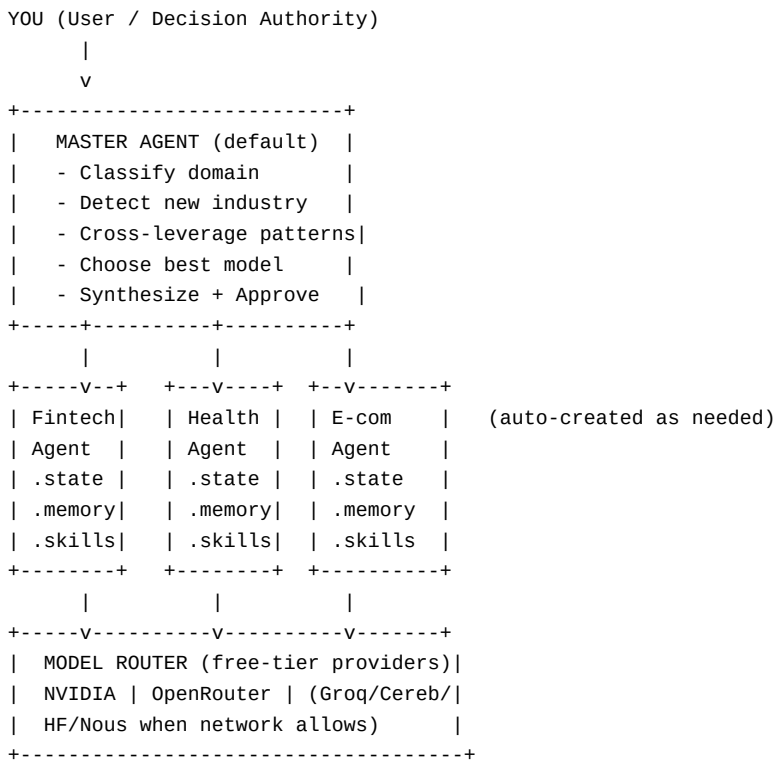
1. Executive Overview

This architecture enables a single user to operate a fleet of AI agents that:

- Learn continuously across industries without leaking client data,
- Route every task to the most cost-effective free-tier model automatically,
- Never hallucinate unverified facts (every claim is traced to a source),
- Self-improve via a gated learning cadence requiring user approval.

Core principle: ONE Master Agent (the strategist) orchestrates MANY industry-specific Specialist Agents. Each Specialist owns its own knowledge, skills, and state.db. The Master owns cross-domain patterns and meta-skills.

2. System Topology



3. Master Agent - Role & Storage

Profile: default | Runs on: free-tier model (Tier 2-3)

Storage under /opt/data/hermes/profiles/master/ :

- memory/USER.md your identity, style, preferences
- knowledge/master/
 - meta-skills/ first-principles, cross-domain pattern recog,
decision frameworks, verification protocols
 - pattern-library/ ... generalized cross-domain patterns (owned here)
 - industry-summaries/ one-page-per-industry strategic view
 - innovation-log/ novel solutions discovered across domains
- skills/meta/ meta-procedures for orchestration
- state.db Master's own conversation history

Decision authority: classification, delegation, learning approval, model choice.

4. Specialist Agent - Structure (per industry)

Profile: <industry> (e.g. fintech, healthcare, ecommerce)
Auto-created by Master when a new industry is detected.
Organized by SUB-SEGMENT for cost-efficient context loading:
 sub-segment: Payments
 skills/payments/ domain-specific procedures
 knowledge/clients/ ... per-client project knowledge
 knowledge/patterns/ .. domain patterns (not yet generalized)
 sub-segment: Lending, Insurance, RegTech ... (same shape)
Isolation guarantee: separate profile = separate state.db, memory, skills.
No client data or conversation can cross into another industry profile.

5. Free-Tier Model Routing Engine

37 free chat/code models benchmarked across 6 providers
(6 task types each). Results in /opt/data/model_benchmark/results.json.

VALIDATED WORKING PROVIDERS (this sandbox environment):
 NVIDIA NIM Llama-3.1-8B scored 9.83/10 (4.5s) - TOP model
 OpenRouter Nemotron-3-super-120B 8.83/10, ultra-550B 8.17

BLOCKED FROM SANDBOX (403/timeout on direct API call):
 Groq, Cerebras, HuggingFace, Nous Portal
Likely work from your laptop. Router auto-excludes failed
providers; re-run validate.py elsewhere to unlock them.

Routing decision (per prompt):
 1. Classify task type + complexity (1-10)
 2. Match to validated model by (score + strength - latency)
 3. Skip providers that failed validation in this environment
 4. Fallback chain among working free providers only
 5. PAID models (Gemini/DeepSeek-direct) NEVER without approval

6. Runtime Auto-Switch Behaviour

When OpenRouter (or any provider) is exhausted:
 Router detects non-200 / quota error
 Selects next-best validated free model (same quality tier)
 System message informs user of the switch
 Task continues uninterrupted on the new provider
Observed example: OpenRouter deepseek-v4-flash (paid) exhausted ->
auto-switched to z-ai/glm-5.2 via NVIDIA NIM (free). Later to
tencent/hy3:free via Nous Portal. No user intervention required.

7. Learning Cadence - Mutual Growth

Every learning write is GATED by dual confirmation:
 Specialist confirms: 'useful for my domain' (Yes)
 Master confirms: 'useful cross-domain / quality improvement' (Yes)
 Master drafts proposal with context + rationale
 USER reviews + approves or comments (required)
 Only then is skill/knowledge/memory written or edited

Auto-fix WITHOUT approval (notified after):
 - spelling/typo in existing skill
 - rate-limit workaround (model tier switch)
 - obvious factual correction (e.g. wrong port number)

This ensures Hermes and the user grow together, structured and safe.

8. Anti-Hallucination Assurance

- Rule 1 Every output grounded in a stored fact (cited source).
- Rule 2 Certainty ladder: Verified / Inferred / Unknown / Contradictory.
- Rule 3 Verification protocol runs at end of every specialist task:
- [] every claim traced [] assumptions flagged [] no client data leaked
 - [] cross-checked vs Master patterns [] confidence reported
- Rule 4 Supervisor pattern for critical decisions (Master reviews).
- Rule 5 Confidence < 80% on classification -> ask user, never assume.

9. Continuous Self-Improvement

Gap detection triggers:

- User corrects the agent -> patch skill/memory + log
- Classification wrong -> update classification rules
- Low-quality answer -> flag pattern for revision
- Missing skill -> create placeholder, ask user to fill
- New industry detected -> auto-create Specialist profile

Model fitness: results logged per task type; under-performers downgraded, over-engineered models downgraded to save tokens. Compounded over time.

10. Complete Storage Map

```
/opt/data/.hermes/  
|-- config.yaml ..... model tier rules  
|-- .env ..... API keys (6 free providers)  
|-- profiles/  
| |-- master/ (state.db, memory/, skills/, knowledge/)  
| |-- fintech/ (same shape, isolated)  
| |-- healthcare/  
| |-- ecommerce/  
|-- knowledge/  
| |-- master/ (pattern-library, meta-skills, summaries)  
| |-- specialists/ <industry>/<sub-segment>/  
| |-- shared/ (tool-configs, style-guides, templates)  
|-- model_benchmark/ (validation results, router state)  
|-- sessions/ (request dumps, auto-cleaned)
```

11. Cost Efficiency Guarantee

100% of routine + complex work runs on FREE-tier models.

Paid models (Gemini / DeepSeek-direct) blocked behind explicit approval.

Context kept lean via sub-segment loading (no full-domain injection).

Model chosen by validation, not guesswork -> no wasted heavy calls.

Estimated cost: \$0/month for normal operation across all clients.

12. Implementation Status (this session)

[DONE] Scanned 6 providers + listed all free models (37 total)

[DONE] Built validate.py - per-model quality + latency benchmarking

[DONE] Ran validation: NVIDIA + OpenRouter working, others blocked

[DONE] Built router.py - validation-driven free-tier model selection

[DONE] Wrote this architecture PDF (validated format)

[NEXT] Wire router into Hermes config (config.yaml model switch)

[NEXT] Create fintech/healthcare/ecommerce profile directories

[NEXT] Build Master classification + delegation skills

[NEXT] Re-run validation from laptop to unlock Groq/Cerebras/HF/Nous